



University of Padua

September 8th, 2026

REAL-TIME KERNELS AND SYSTEMS

A study of "An Empirical Study of Performance Interference: Timing Violation Patterns and Impacts"

Lorenzo Albertin
Andrea Santi
Caterina Vallotto

• <u>Interpretation of the exam requirements</u>	<u>3</u>
• <u>Further insights into A-E3-50</u>	<u>10</u>
○ <u>ArduPilot overview</u>	<u>11</u>
○ <u>(1) Goal</u>	<u>15</u>
○ <u>(2) Expectations</u>	<u>16</u>
○ <u>(3) What happened - Empirical evidence</u>	<u>17</u>
○ <u>(4) Why</u>	<u>18</u>
○ <u>(5) Impacts on the results</u>	<u>21</u>
○ <u>(6) Generalization</u>	<u>22</u>
• <u>Learning outcomes</u>	<u>23</u>
• <u>Self criticism</u>	<u>28</u>
• <u>Teaching and exam procedures</u>	<u>29</u>

EXAM REQUIREMENTS AND LEARNING OBJECTIVES

- Critical understanding of the paper
- Familiarize with the literature background to the paper's research
- Define our learning objectives
 - a. Understand the relation between timing violations, data freshness, and control degradation.
 - b. Compare the effects of different perturbation mechanisms and injection locations.
 - c. Analyze the propagation of timing violations to control outputs and physical behavior.
- Reproduce – in the small – the original experiments and results

2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS) | 979-8-3503-5841-4/24/\$31.00 ©2024 IEEE | DOI: 10.1109/RTAS61025.2024.00033

2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)

An Empirical Study of Performance Interference: Timing Violation Patterns and Impacts

Ao Li, Jinwen Wang, Sanjoy Baruah, Bruno Sinopoli, Ning Zhang
Washington University in St. Louis
{ao, jinwen.wang, baruah, bsinopoli, zhang.ning}@wustl.edu

Abstract—Multi-core platforms are becoming increasingly prevalent in cyber-physical systems such as automobiles and robots. However, contention for shared resources makes it challenging to guarantee timing predictability. Existing studies have primarily focused on characterizing the extent to which such interference can induce delays (usually from an adversarial perspective). Unfortunately, less is understood on the physical impacts of these timing delays in different cyber-physical platforms. In this paper, we fill this gap by providing an empirical examination of the end-to-end effects of performance interference on real-world applications. We analyze the root causes of harmful interference and summarize potential implementation pitfalls.

To automate this process, we introduce TimeTrap, a tool that analyzes performance interference in autonomous systems through the lens of control outcome. To understand the extent to which timing interference may cause control deviations, TimeTrap has to strategically leverage different magnitudes of resource contention to trigger targeted deadline miss patterns. Through this exercise, we found that a naïve approach that maximizes task latency via performance interference may fail to trigger worst-case outcomes (i.e. physical damages) due to built-in fail-safe mechanisms. As a result, delays have to be induced in a stealthy manner to avoid triggering fail-saves. To achieve this, TimeTrap first employs a system that actively injects fine-grained delays into the target software, adjusting the duration based on measured feedback. Second, TimeTrap leverages predictability in CPS execution patterns and resource usage to automatically tune its aggressor workloads, matching these patterns to achieve targeted interference and execution delays in a victim. We evaluate TimeTrap on two physical-world platforms and six platforms in a hardware-in-the-loop simulation environment, including robotic arms, UGVs, UAVs, self-driving cars, and humanoid robots. These studies demonstrate that an interference-based attack surface exists in different stages of the CPS pipeline, from perception to planning and control.

I. INTRODUCTION

Multi-core architectures are highly effective at providing increased computational resources under constraints on size, weight, and power (SWaP) in embedded platforms. However, when deployed in real-time cyber-physical systems (CPS) where timing predictability is essential, performance interference from contention for shared resources among concurrent tasks is increasingly a concern. Such interference may lead to delays in data propagation along the processing pipeline in modern CPS platforms, consequently degrading performance.

Existing Literature. Understanding the impacts of performance interference in CPS requires a comprehensive analysis of both the sources and characteristics of interference, as well as its resulting impact on control outcomes. This demand is

also highlighted by a recent industrial challenge presented by Arm [1]. Existing studies follow three distinct lines of research: (i) The first exclusively examines the extent of worst-case performance interference [2]–[12] without considering the subsequent implications on system and software behavior. (ii) The second primarily focuses on analyzing the propagation of delays throughout the execution pipeline [13]–[19]. Such works approach the problem from the scheduling perspective, yet often do not consider the impact on control outcomes. (iii) The third abstractly models the control degradation resulting from various deadline miss patterns in an individual task [20]–[23]. However, these models do not consider the broader view of cascading effects among multiple tasks in the system. To analyze the impact of performance interference through a holistic view that interconnects software properties, real-time properties, and control properties [24], our work empirically studies the manifestation of performance interference in different CPS platforms to illuminate the implications for physical safety risks in the real world.

Systematizing Timing Issues in Real-World CPS Software. To develop an appropriate abstraction over timing issues, we analyzed 241 real-world timing-related bugs from eight prominent CPS applications. We discovered that these bugs often arise from the inadequate specification or enforcement of timing semantics. We also found that not all timing bugs result in adverse control outcomes in the physical world. In this paper, we use *timing issues* to refer to the timing bugs that produce adverse physical outcomes. Dissecting these real-world timing issues has provided us with new insights to formulate the problem of performance interference impact as *temporal displacements* in key state variables of the CPS process (more details about the bug systematization can be found in Section II).

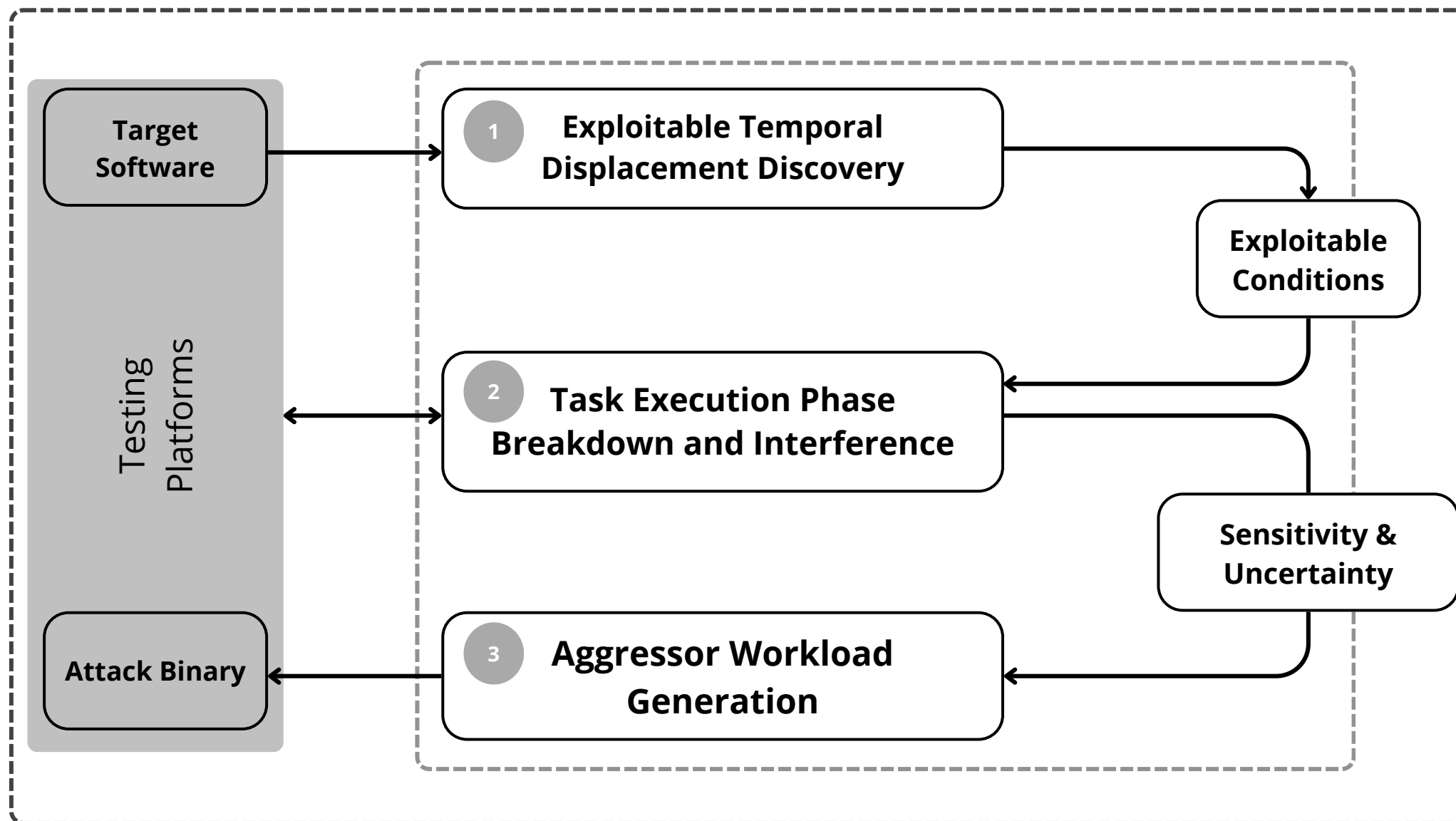
TimeTrap — Technical Challenges and Solutions. Informed by our observations, we introduce *TimeTrap*, a tool that aims to characterize the worst-case end-to-end impacts of performance interference by adversarially inducing interference with the goal of maximizing control deviation (i.e. difference in physical outcome). However, naïvely exhausting system resources is often ineffective due to built-in safety checks, as we discuss in Section II. Consequently, it is imperative to discern which task(s) should be delayed in order to effect significant control deviation. Determining how much delay the attacker should cause is also challenging, since too little

2642-7346/24/\$31.00 ©2024 IEEE
DOI 10.1109/RTAS61025.2024.00033
Authorized licensed use limited to: POLO BIBLIOTECARIO DI INGEGNERIA. Downloaded on May 13,2026 at 06:43:30 UTC from IEEE Xplore. Restrictions apply.

320

OUR INTERPRETATION OF “REPRODUCE IN THE SMALL”

TimeTrap framework



1

Find the damaging timing pattern

Where and how much delay causes control degradation?

2

Map interference to delay

Which resource and intensity can reproduce that timing pattern?

3

Generate targeted interference

Reproduce the harmful delay while minimizing effects on other tasks.

OUR INTERPRETATION OF “REPRODUCE IN THE SMALL”

Our reproduction

A small-scale experimental reproduction of some of the phenomena that TimeTrap allows us to observe

TimeTrap

Automated mechanism to identify vulnerable regions of code

Search-and-optimization process to derive the perturbation values

Task profiling and sensitivity analysis to automatically generate optimized workloads

Our work

Manual selection of delay injection points

Manually controlled delay magnitude

Simplified aggressor workloads

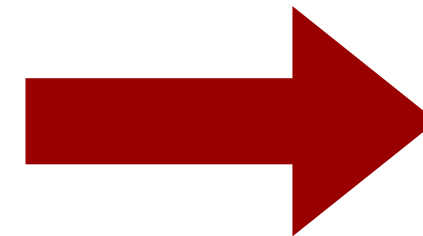
OUR INTERPRETATION OF “REPRODUCE IN THE SMALL” – WHAT WE ANALYZED?



Relationship between a time-domain *perturbation within the system*, the *resulting time displacement or data aging*, and *any degradation in control performance*

Focus on timing-to-control propagation:

1. Introduce a perturbation
2. Observe the internal temporal effect
3. Observe potential effect on control



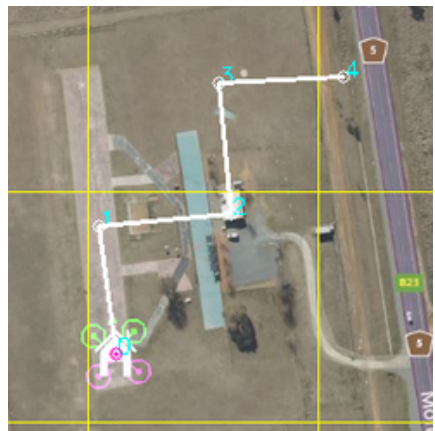
internal temporal anomaly



external failure

MAIN QUESTION AND CASE STUDIES

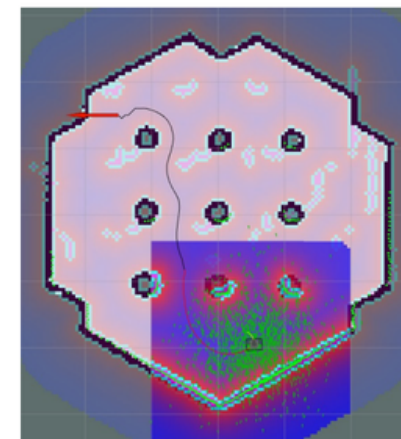
“ If we introduce a controlled temporal perturbation into a Cyber-Physical System, when does it remain confined to the perturbed component, and when does it instead produce a temporal misalignment that can propagate to the control behavior? ”



ARDUPILOT

How is the temporal perturbation produced?

- Indiscriminate saturation
- Software delay injection
- Low-priority cache-contention aggressor



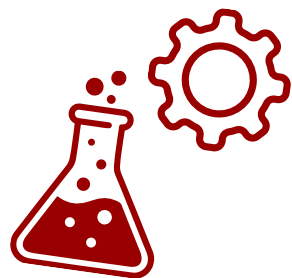
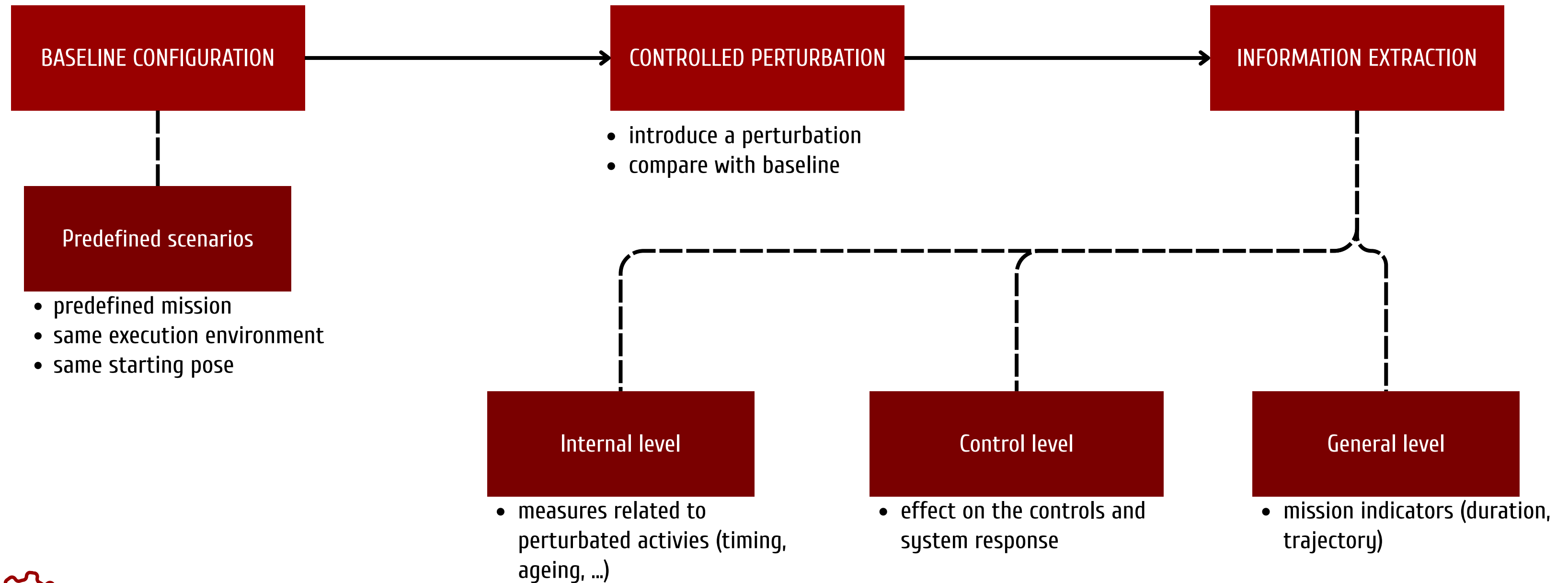
TURTLEBOT3

Where is the temporal perturbation introduced?

- Before the publication of the AMCL pose estimate
- Before the update of the local costmap
- Beginning of the DWA controller
- ...



EXPERIMENTAL APPROACH



GENERAL OBSERVATIONS

1

Observing a temporal displacement in a component does not automatically mean observing an equivalent degradation at the mission level

2

The system's behavior cannot be described solely through a binary classification of mission completed versus mission failed

3

The timing effect of resource contention depends on the victim's sensitivity to the selected contention channel.

A temporal perturbation can produce a measurable internal anomaly

Possibilities of end-to-end effect

It depends on

where it is introduced

its intensity

interdependencies
among the components

the ability of the rest of the
system to temporarily absorb
that anomaly

MAIN ASPECT

“How can the sensor producer continue to provide up-to-date information while the waypoint controller uses a previously processed state that has since become less current, and how can this temporal misalignment affect the drone's control behavior?”

ARDUPILOT OVERVIEW

SITL (Software-In-The-Loop) allows flight testing without a physical drone. ArduPilot runs the flight-control software, while a virtual vehicle model simulates the drone's physical motion and produces the corresponding sensor data.

Wall-Clock Time

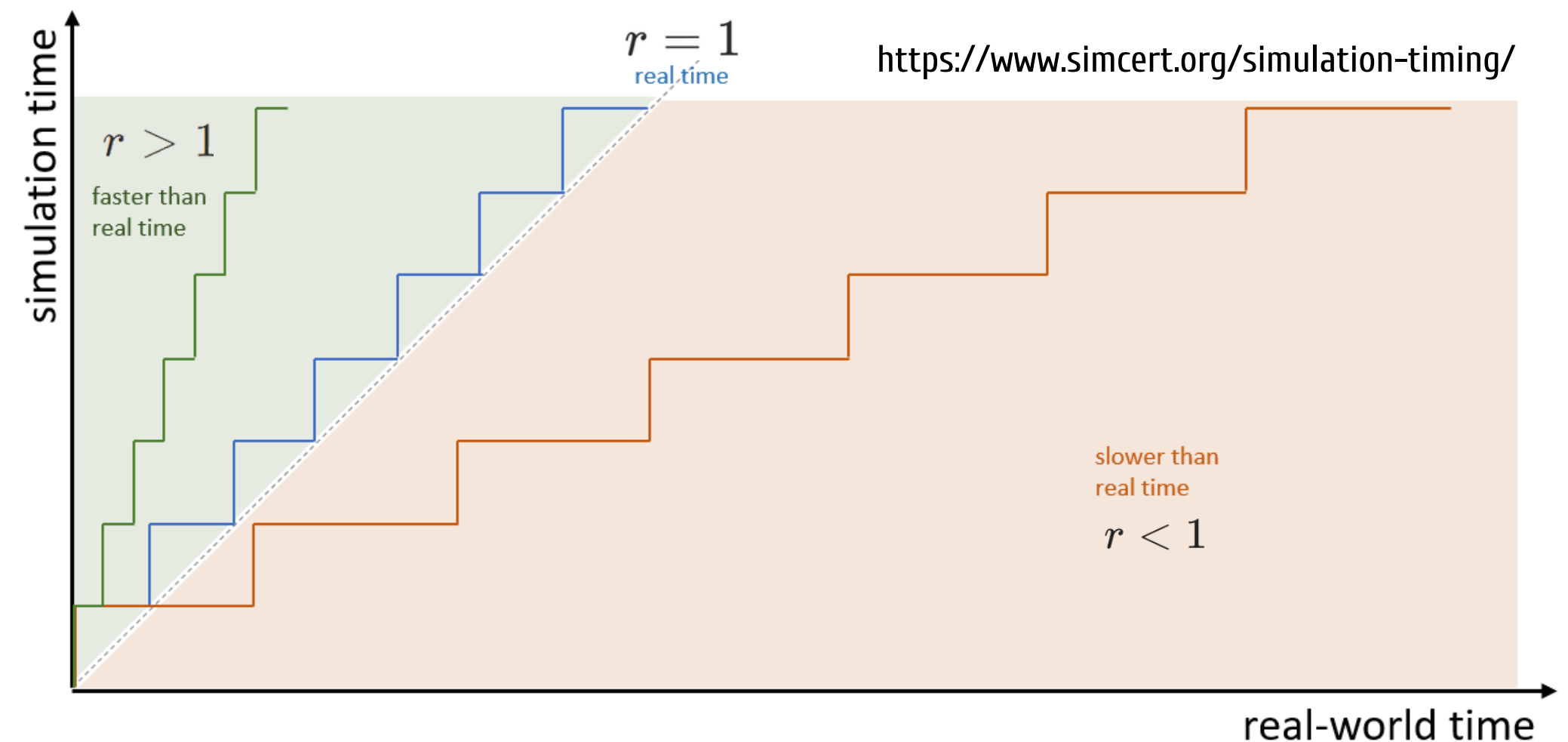
Real-time elapsed in the virtual machine

Simulation Time

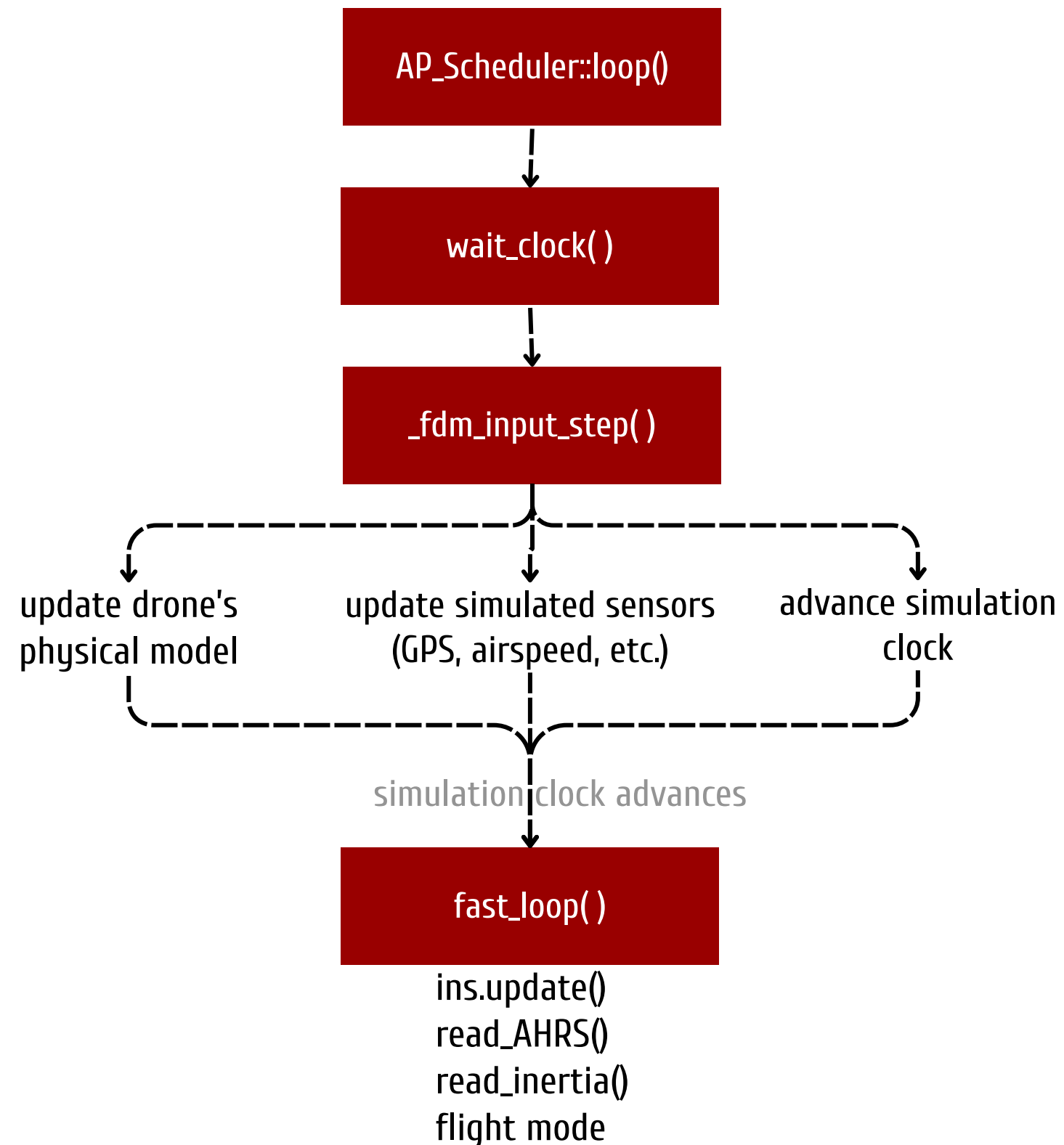
Internal time elapsed in the simulated world

Advances with the progression of the *flight-dynamics model*

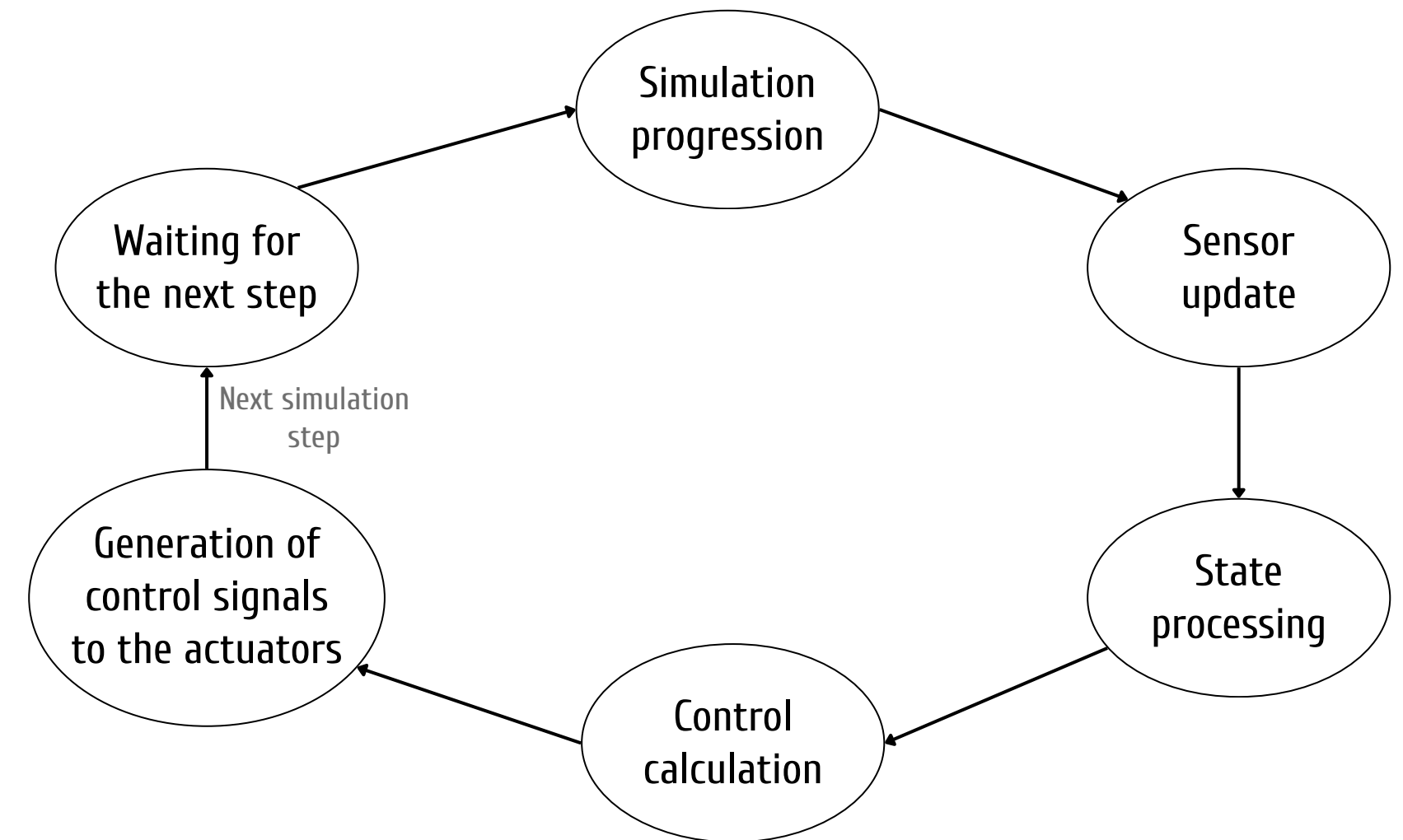
$$r = \frac{\Delta t_{sim}}{\Delta t_{wall}}$$



ARDUPILOT OVERVIEW - ORIGINAL ARCHITECTURE

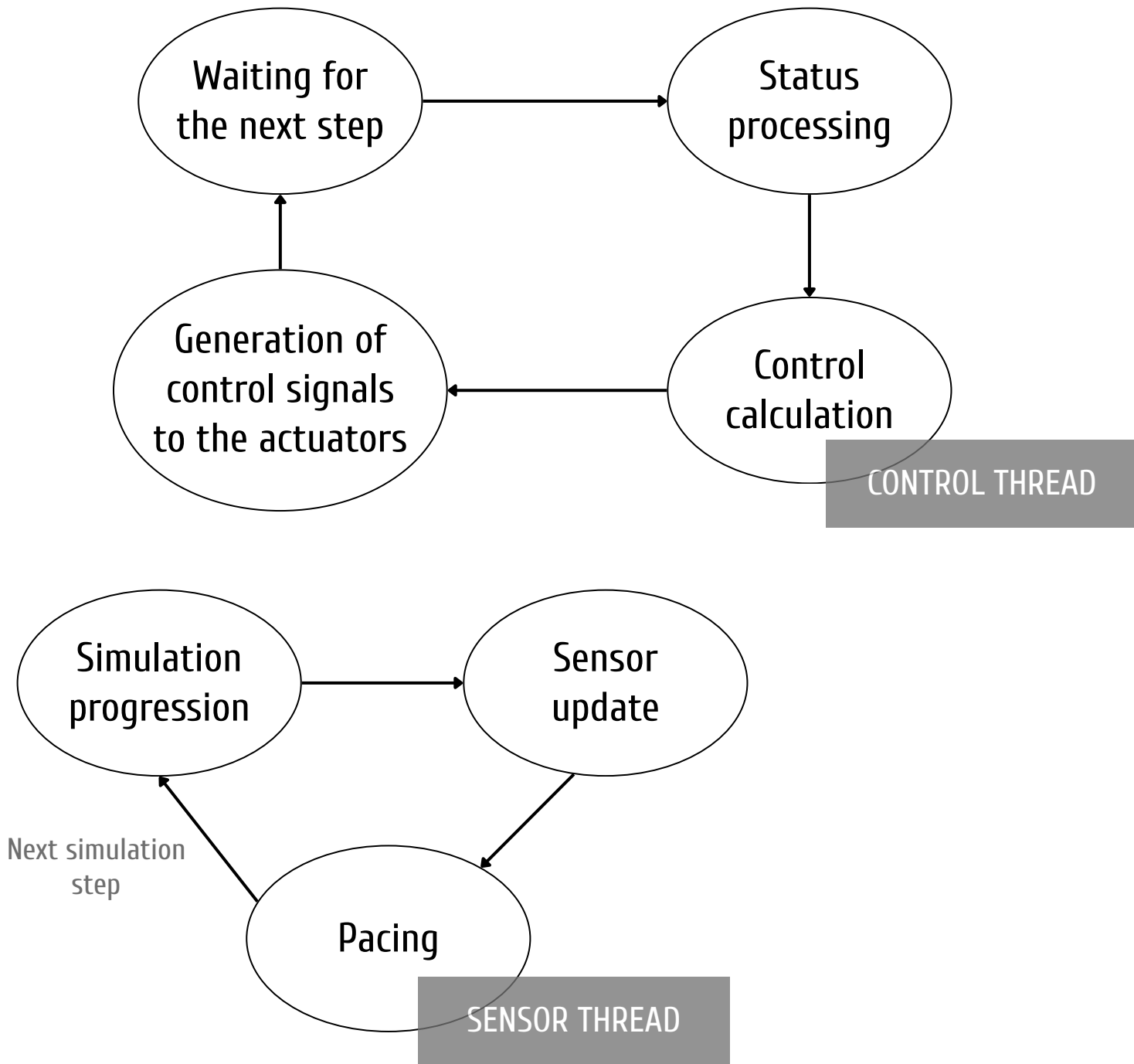
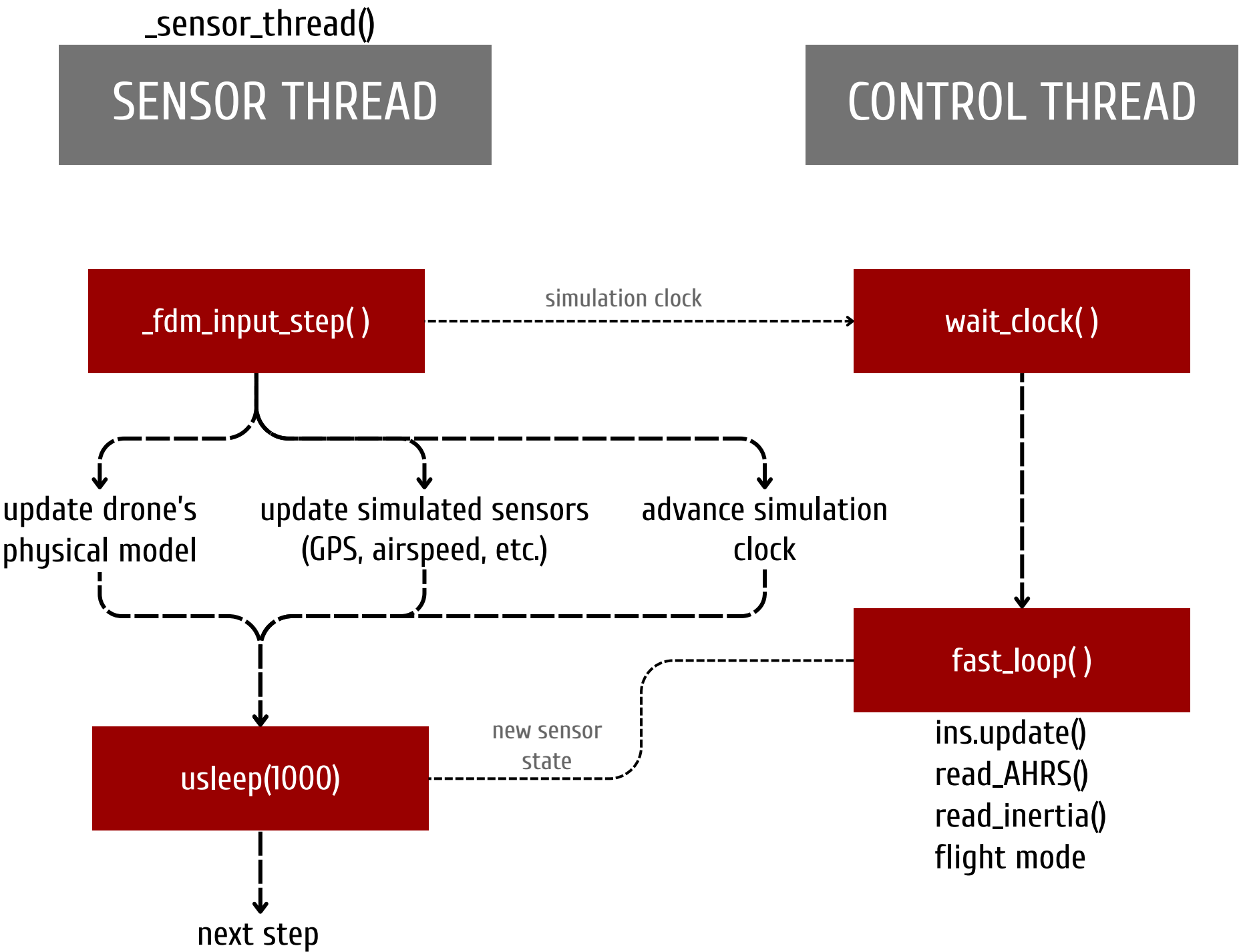


Physics/sensor production and control execution belong to the same execution flow



ARDUPILOT OVERVIEW - MODIFIED ARCHITECTURE

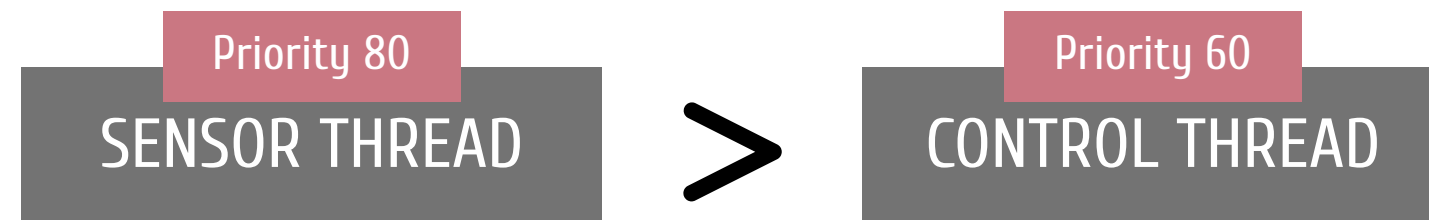
Sensor/physics production and control execution can progress independently.
Simulation progression is moved to an independent sensor thread, so physics/sensors can advance while the control thread is delayed.



ARDUPILOT OVERVIEW – MODIFIED ARCHITECTURE

Priority – FIFO Scheduling

When runnable threads compete for the same CPU, the one with the highest priority is selected.

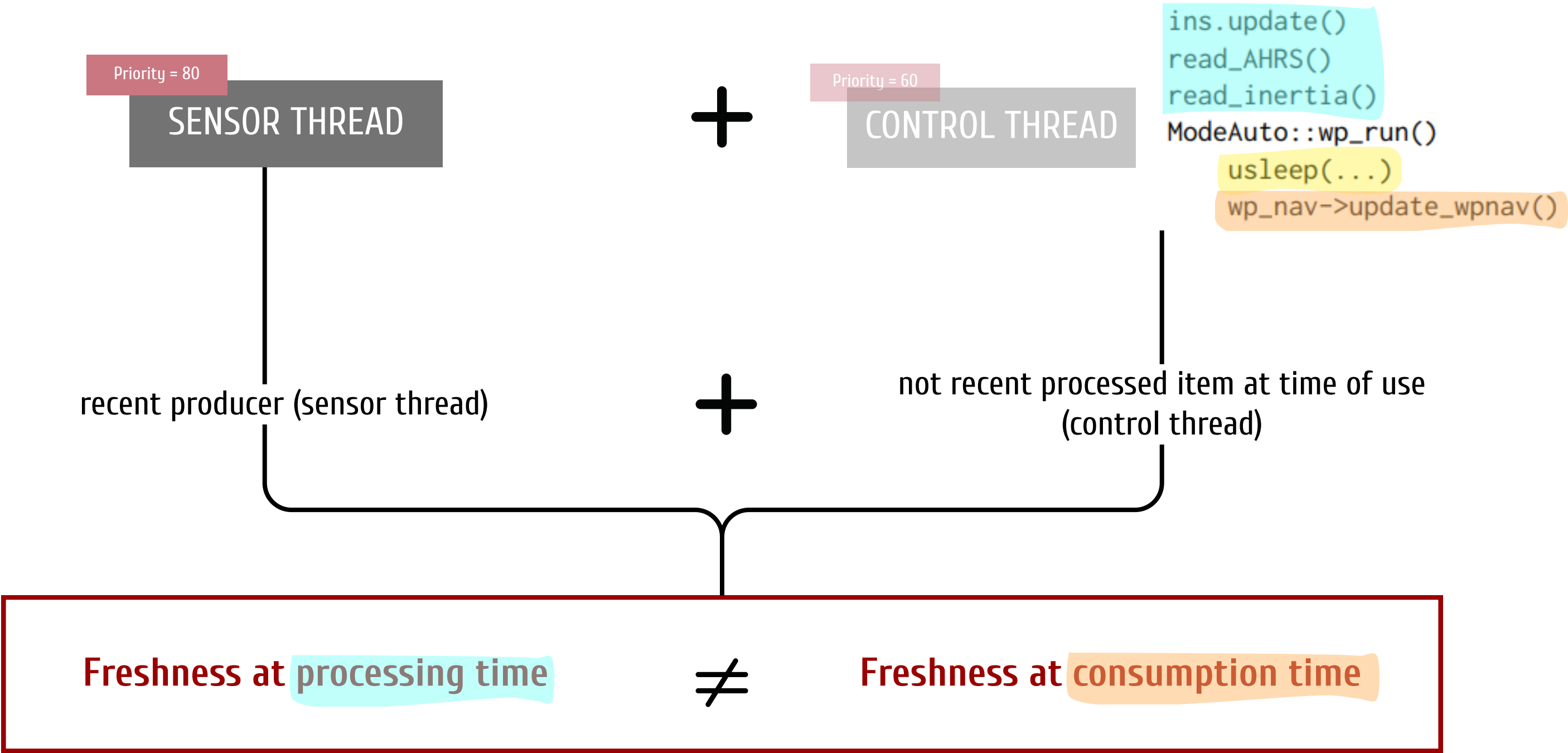


Pacing Mechanism

To prevent the sensor thread from continuously calling `_fdm_input_step()`.



(I) GOAL



And what about
the end-to-end
propagation at
mission level?

(2) EXPECTATIONS

1

At the end of the suspension, very recent information from the producer might be available in the system, while 'wp_nav' would continue to use the state that the control thread had processed before the suspension

```
ins.update()  
read_AHRS()  
read_inertia()  
ModeAuto::wp_run()  
    usleep(...)  
wp_nav->update_wpnav()
```

2

The combination of "low producer age + high temporal displacement" indicates that recent updates are available, while the controller is using an older processed state.

$$A_{producer} \approx 0.040 ms \quad (\text{baseline A-E2})$$

$$TD_{state} \gg 0.171 ms \quad (\text{baseline A-E2})$$

3

The waypoint controller calculates its action using a representation of the vehicle based on a previous point in time relative to the state that the simulated world had already reached



(3) WHAT HAPPENED - EMPIRICAL EVIDENCE

A-E2 (baseline of modified architecture)

$$TD_{state} = 0.171 \pm 0.011 \text{ ms}$$

$$A_{producer} = 0.046 \pm 0.007 \text{ ms}$$

$$RMSE_{200} = 24.58 \pm 0.23 \text{ cm/s}$$

$$RMSE_{NAV3} = 48.57 \pm 0.14 \text{ cm/s}$$

$$e_{max_NAV3} = 104.80 \pm 0.81 \text{ cm/s}$$

A-E3-50 (50 ms delay injection)

$$TD_{state} = 28.175 \pm 0.389 \text{ ms}$$

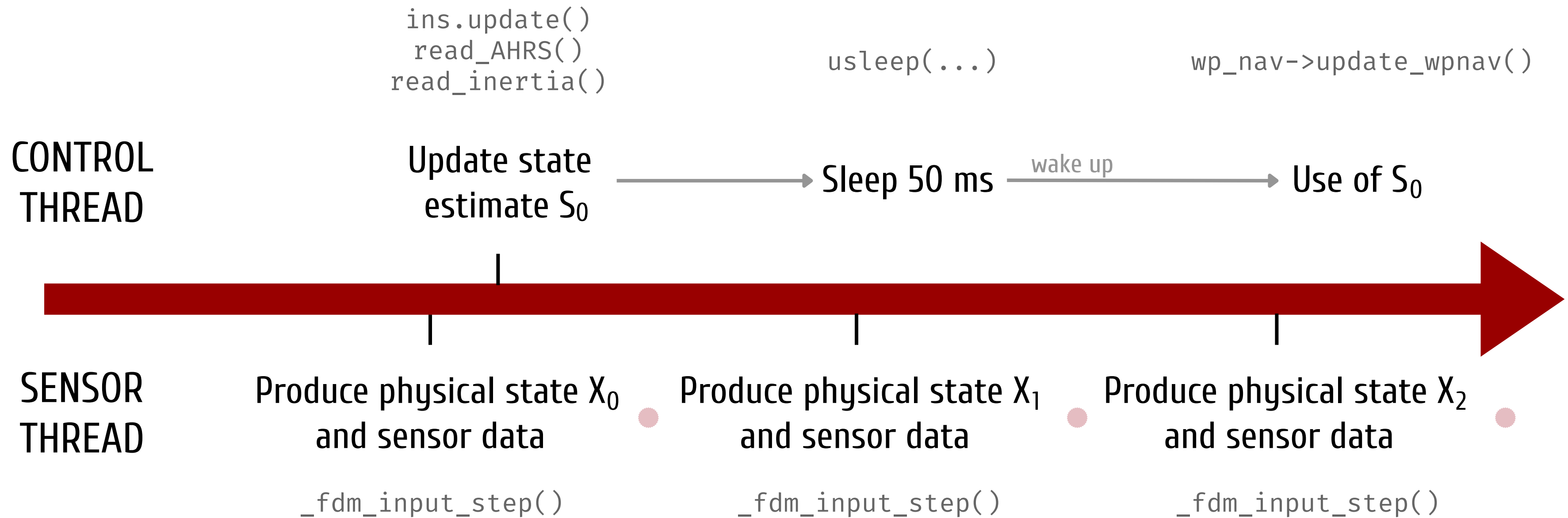
$$A_{producer} = 0.040 \pm 0.015 \text{ ms}$$

$$RMSE_{200} = 115.50 \pm 8.34 \text{ cm/s}$$

$$RMSE_{NAV3} = 193.03 \pm 40.49 \text{ cm/s}$$

$$e_{max_NAV3} = 810.42 \pm 138.30 \text{ cm/s}$$

(4) WHY - FRESHNESS AT PROCESSING TIME $\neq$ FRESHNESS AT CONSUMPTION TIME



(4) WHY - 50 ms OF SLEEP → 28 ms OF TEMPORAL DISPLACEMENT

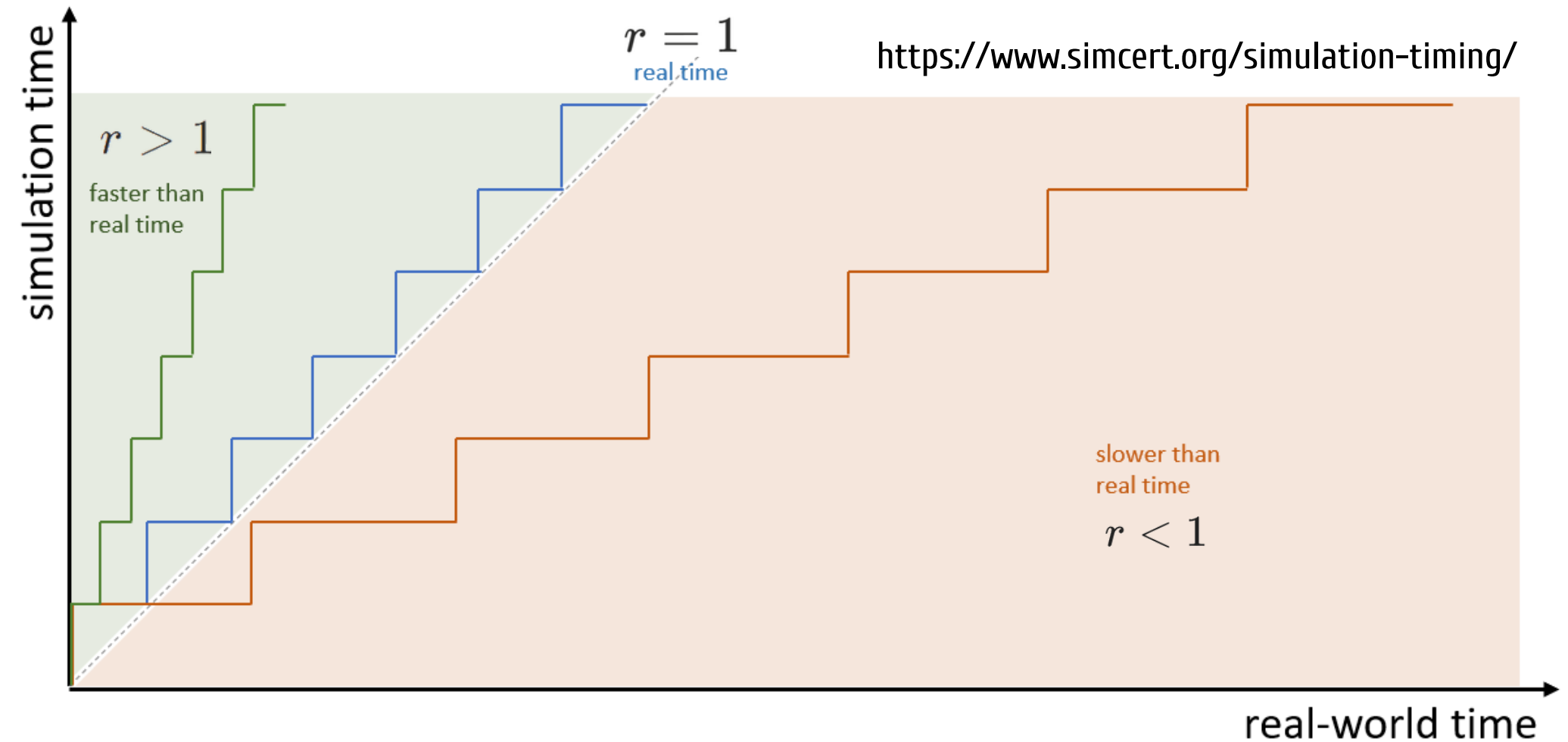
$$D_{wall} = 133.85s$$

$$D_{sim} = 74.78s$$

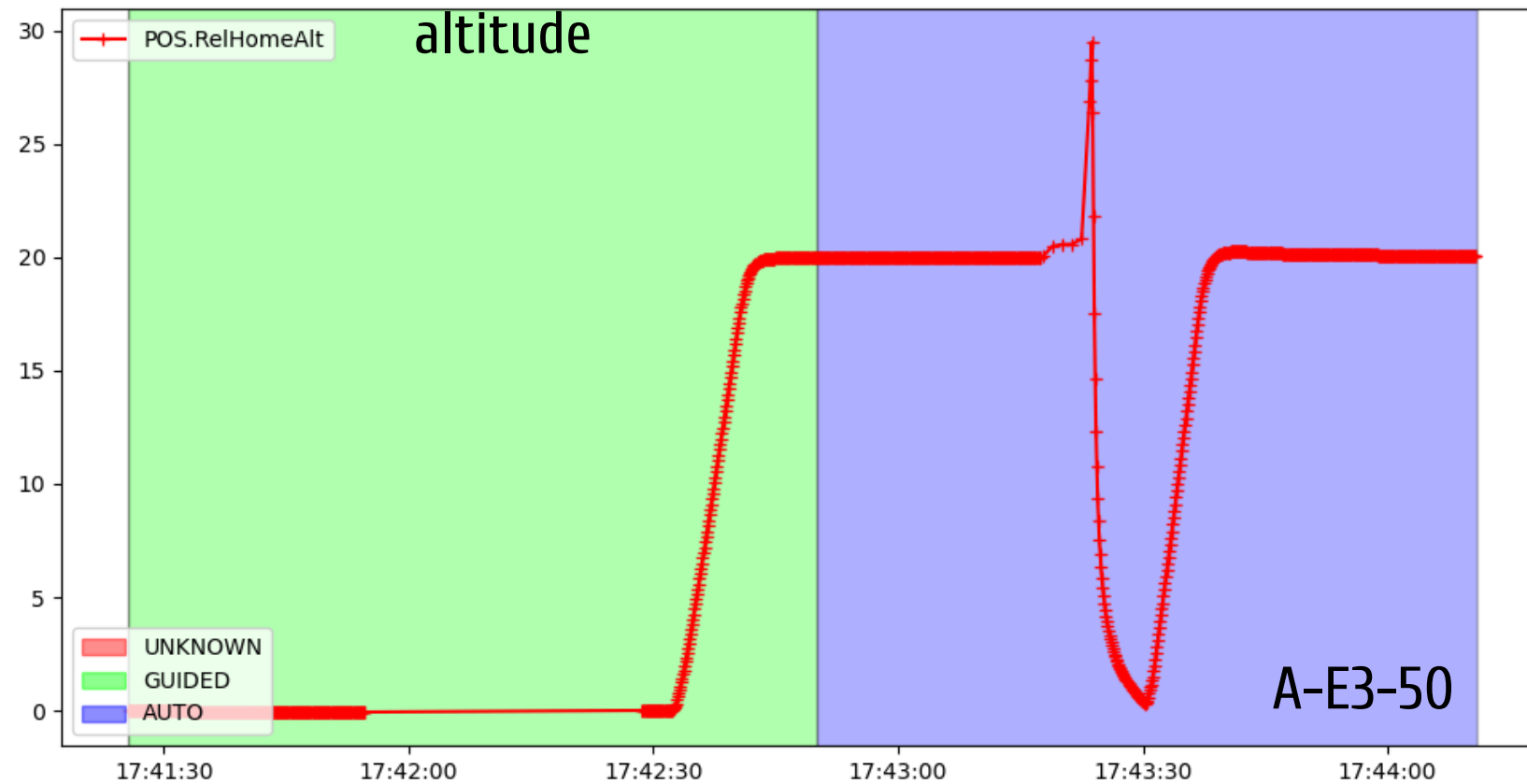
$$\frac{D_{sim}}{D_{wall}} \approx 0.56$$

$$50 \cdot 0.56 \approx 28ms_{sim}$$

$$TD_{state} = 28.175 \pm 0.389 ms$$



(4) WHY – MISSION LEVEL PROPAGATION



- loss of state freshness
- commands calculated during the perturbation
- trajectories and deviations (altitude-velocity) different from nominal values
- error persists even after the delay
- recovery phase

$$RMSE_{NAV3} = 48.57 \pm 0.14 \text{ cm/s}$$

$$RMSE_{NAV3} = 193.03 \pm 40.49 \text{ cm/s}$$

$$e_{max_NAV3} = 104.80 \pm 0.81 \text{ cm/s}$$

$$e_{max_NAV3} = 810.42 \pm 138.30 \text{ cm/s}$$



(5) IMPACTS ON THE RESULTS

- 1 We can distinguish between a simple delay in the control thread and an actual loss of state freshness at the point of consumption.
- 2 An internal time anomaly propagates to the control behavior.
- 3 "Mission success" is not a sufficient metric.

(6) GENERALIZATION

What can be generalized?	What cannot be generalized?
Independent producer/consumer can become temporally misaligned	50 ms wall → 28 ms state age
Delaying a consumer after state processing but before use can make the consumed state stale	28 ms age → 193 cm/s RMSE & 810 cm/s max error
Injection location is part of the mechanism, not only delay magnitude	Results depend on VM, mission, vehicle dynamics and controller
Applicable to other producer–consumer / functional-chain architectures	usleep() is not equivalent to TimeTrap hardware interference

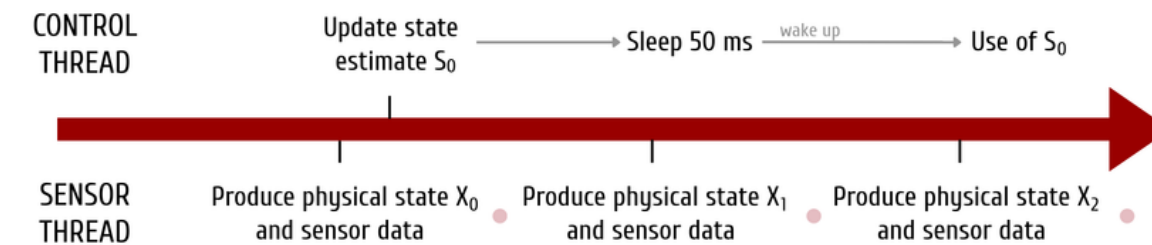
T01

What we learned in class

Correctness → time-and-value domain

A correct value returned later than due can be bad

What we observed in our project



Availability of recent data $\neq$ Controller is processing a recent status

The generation of usable commands by the controller is delayed → Reduced temporal correctness

ArduPilot

TurtleBot3

What information is used, how old it is when it is used, and how long it takes for the generated control command to actually reach the actuation phase

T01 – Dependability

What we learned in class

**The relationship between cause,
abnormal internal state, and external
manifestation**

Failure} → {Fault → Error → Failure} → {Fault

What we observed in our project

Delay injection → Internal Temporal Displacement → ? → Mission level degradation

Some perturbations remained largely confined, while those introduced closer to the generation of control commands had a much more direct impact on the velocity commands at the actuation stage

TurtleBot3

A temporal perturbation does not automatically imply a failure

T09

What we learned in class

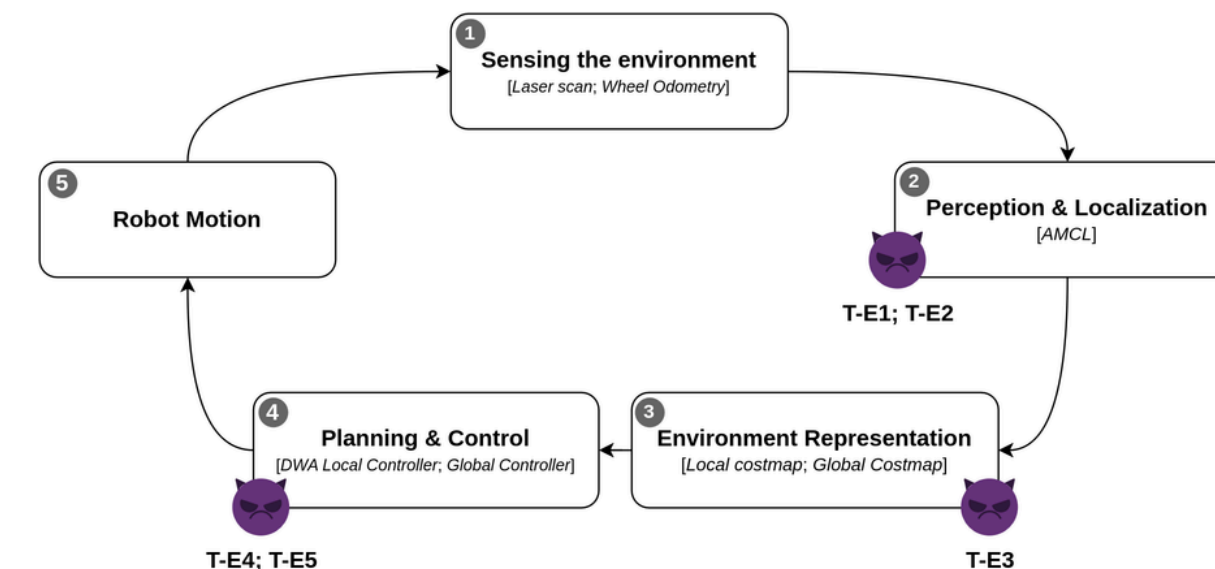
Transactions

a chain in which the completion of one activity enables or influences what the next one can do

What we observed in our project

To understand the effects of the delay injection, it is important to study:

- What happened locally?
- When the next data is generated?
- Which component consumes it?
- Which of the previous information remains available?
- Does the perturbation affect the control output?



TurtleBot3

The magnitude of the delay, on its own, does not describe its impact
It is important to know where the pertubated component is located in the functional chain and what causal dependencies link it to the output

T02 and T03

What we learned in class

Scheduling priority-driven

The scheduler selects from the jobs that are in the ready queue

Fixed-Priority Scheduling

When multiple jobs are competing for the same processor, the one with the highest priority is executed first

What we observed in our project

FIFO Scheduling



+

Pacing Mechanism

Implement SCHED_FIFO in practice and design the pacing strategy while taking into account both parallelism across different vCPUs and contention on the same vCPU.

T11

What we learned in class

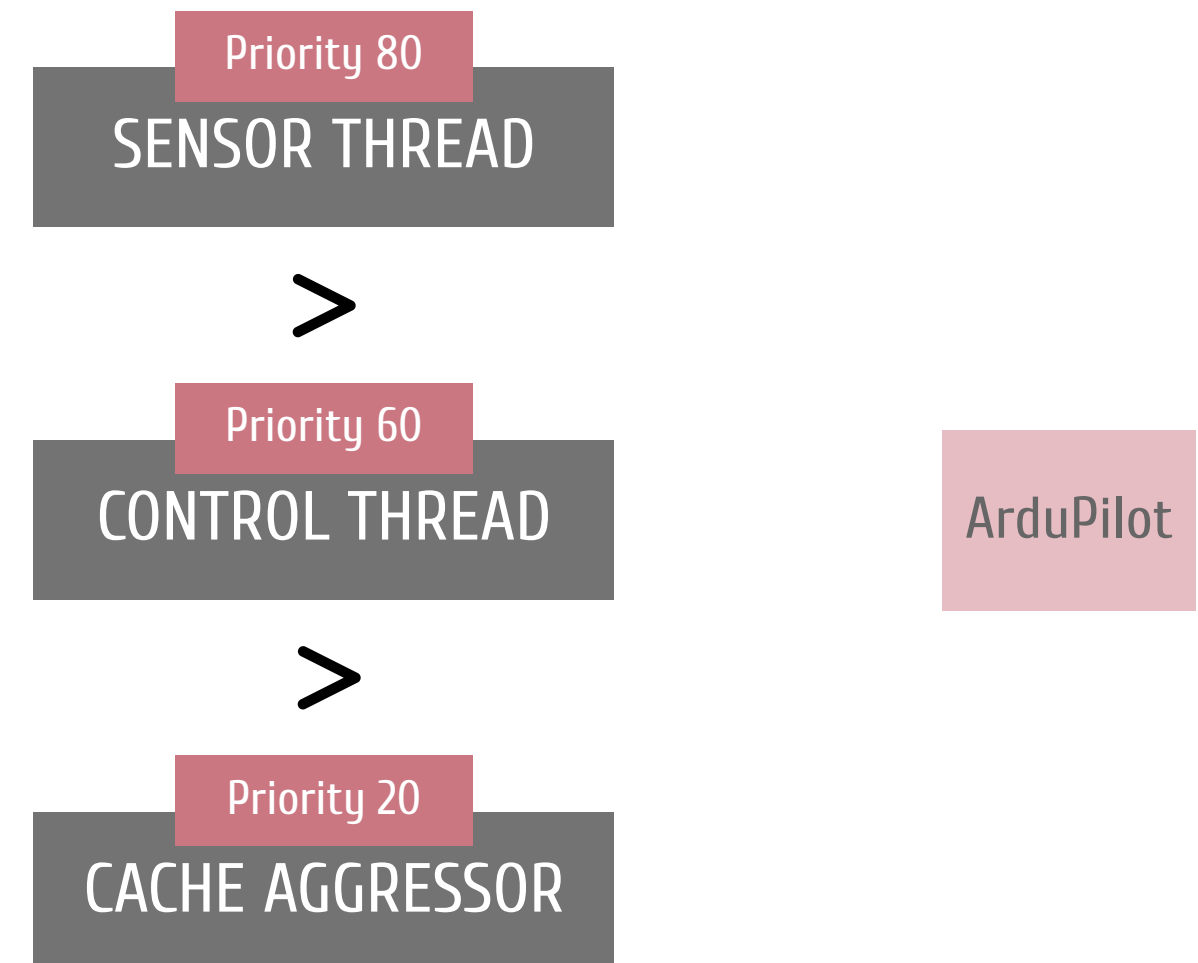
Stalling

A task may be formally running but may not be progressing at its nominal rate because it has to wait for a shared resource



Execution time of a task may also depend on accesses to shared hardware resources generated by tasks running on other cores

What we observed in our project



The presence of a shared resource does not guarantee that an attacker will cause a time-based effect. You need to know: which resource the victim is using, at what stage it is using it, and if the attacker's workload is actually competing for that same resource.

WHAT WENT WRONG



Breadth over depth

We explored many experiments and metrics, but this sometimes prevented us from analysing the most important mechanisms in enough depth.



Experimental approach

We started with a trial-and-error approach and only later defined a clearer and more structured experimental methodology.



Parameter selection

Some experimental parameters were chosen mainly to make the effects observable, rather than being derived from an analytical or systematic preliminary study.

FEEDBACK



From theory to practice

The project helped us turn theoretical RTKS concepts into concrete problems that we had to implement, measure, and interpret.



Active learning during the course

The interactive structure of the lectures encouraged us to study continuously, discuss the topics together, and stay engaged during the course.



Clarifying “reproduce in the small”

A clearer indication of the expected balance between breadth and depth could help students define a more appropriate project scope from the beginning.



Intermediate project checkpoint

An early discussion about the experimental direction could help students identify methodological issues before investing too much time in implementation.



University of Padua

Thank you for your attention

Lorenzo Albertin
Andrea Santi
Caterina Vallotto